DEEP LEARNING FOOD ANALYSIS

Cesaratto Ilaria - Porro Andrea

Research Questions and Problems

RESEARCH QUESTION

Can transfer learning from ImageNet-pretrained networks effectively compensate for a small, imbalanced training set, and which adaptation strategy performs best under these constraints?

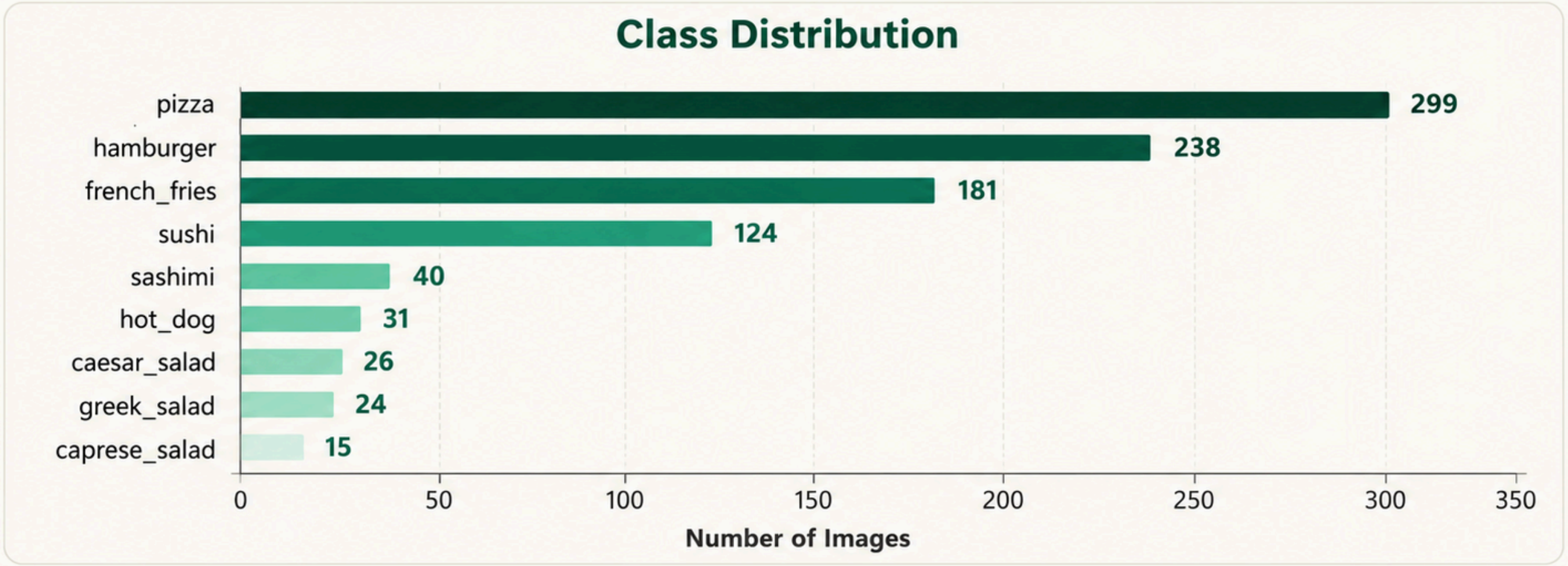
OBJECTIVES

1. Compare transfer learning with a CNN trained from scratch
2. Compare feature extraction with fine-tuning
3. Compare models with frozen deep features + classical ML

MAIN CHALLENGES

- Class imbalance
- Small dataset

Exploratory Data Analysis



Dataset:
9 classes (pizza, hamburger, french fries, sushi, sashimi, hot dog, caesar salad, greek salad, caprese salad).
978 images total. Image sizes range from 59×64 px to 3700×4300 px.

Imbalance:
the majority class is about 20 times larger than the minority class.

Implication:
this imbalance can bias training and must be addressed in model design.

Data Cleaning and Data Leakage Prevention

978 initial images

-35 manual review: problematic images removed

4 mislabeled images relabeled

-46 exact duplicates removed

897 final images

Group-aware + stratified split: **70 / 15 / 15**

Train 70% (augmented) • Val 15% • Test 15%

Near-duplicate discovery

Exact duplicate removal was not enough: some images were visually very similar, with only small changes in framing, angle, or context. We grouped these similar samples before splitting the dataset.

Example of near-duplicate images

pizza_174.jpg

pizza_185.jpg

These two images differ only slightly, so they belong to the same similarity group.

Traditional split — risk of data leakage

Near-duplicate images may be scattered across train, validation, and test. The model may then recognize almost the same image instead of truly generalizing.

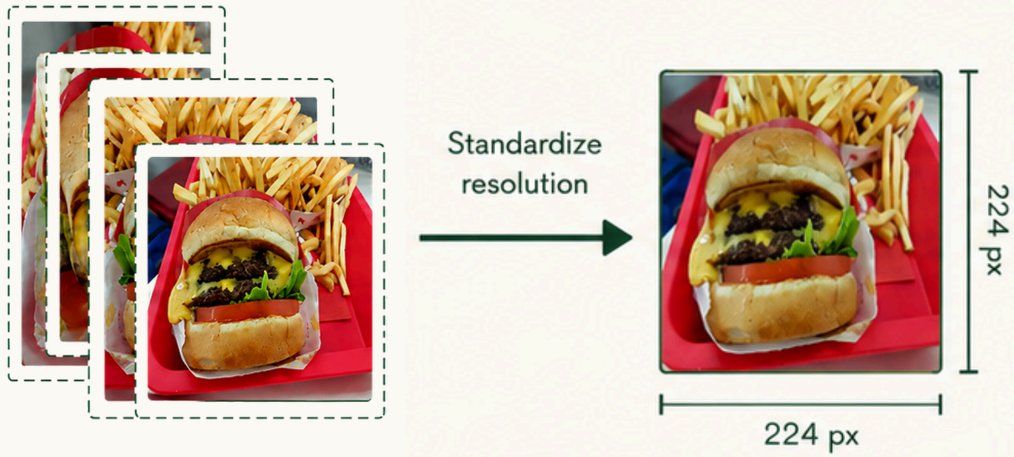
Group-aware + stratified split

Each similarity group is assigned to one split only, while class proportions are preserved. For example, the two pizza images above stay in the same split, preventing leakage.

Preprocessing & Data Agumentation

1 Resize to 224 × 224

The dataset contains images with heterogeneous sizes. Therefore, all images were resized to a fixed input resolution of 224 × 224 pixels before entering the models.



2 Augmentation only on train

original

augmented

Applied augmentations:

- Horizontal flip
- ±10% rotation / zoom
- ±15% brightness / contrast
- No hue or saturation shifts

Augmentation was intentionally kept realistic and conservative: food categories should remain visually plausible. We avoided transformations that would alter the natural appearance of the dish, especially color-related changes.

The goal was to improve robustness without changing the semantic identity of the food.

3 Modeling Strategies

Preprocessing

EfficientNetV2S: pixels in [0, 1]

ResNet50: ImageNet channel-wise preprocessing (BGR mean subtraction)

MobileNetV3Large: pixels in [-1, 1]

CNN from scratch: [0, 1] normalization

Baseline CNN
from scratch

- Learns visual features directly from the dataset
- Custom shallow CNN
- Needs more data to generalize well

Transfer Learning
3 pretrained backbones

- EfficientNetV2S
- ResNet50
- MobileNetV3Large

Classical ML
on frozen deep features

- Frozen embeddings extracted from a pretrained network
- No end-to-end neural fine-tuning
- Classifier trained on top of deep features

SVM • **Random Forest** • **XGBoost**

Imbalance handling

Class imbalance was handled through class weights inversely proportional to class frequency.
To avoid excessive emphasis on rare classes, the weights were capped at a **maximum of 5.**

Baseline: CNN from Scratch

INPUT IMAGE

224 × 224 × 3

CONV BLOCK 1
16 filters

112 × 112 × 16

Conv2D (3×3)
+ BatchNorm
+ LeakyReLU
+ MaxPool
+ SpatialDropout2D (0.2)

CONV BLOCK 2
32 filters

56 × 56 × 32

Conv2D (3×3)
+ BatchNorm
+ LeakyReLU
+ MaxPool
+ SpatialDropout2D (0.2)

CONV BLOCK 3
64 filters

28 × 28 × 64

Conv2D (3×3)
+ BatchNorm
+ LeakyReLU
+ MaxPool
+ SpatialDropout2D (0.2)

CONV BLOCK 4
128 filters

14 × 14 × 128

Conv2D (3×3)
+ BatchNorm
+ LeakyReLU
+ MaxPool
+ SpatialDropout2D (0.2)

CLASSIFIER HEAD

Global Average Pooling → Dense (128) → BatchNorm → LeakyReLU → Dropout (0.5) → Softmax (Output)

LEGEND

- Convolutional Feature Maps
- Pooling / Dropout
- Dense / Fully Connected
- Output / Softmax

- 4 conv blocks, 16→32→64→128 filters
- Each block: Conv2D (3×3) + BatchNorm + LeakyReLU + MaxPool + SpatialDropout2D (0.2)
- Global Average Pooling → Dense (128) → BatchNorm → LeakyReLU → Dropout (0.5) → Softmax
- Adam (lr 1e-4)
- Categorical cross-entropy with label smoothing (0.1)
- Kept deliberately shallow: small dataset + limited Colab compute

Transfer Learning: Backbones & Two-Phase Strategy

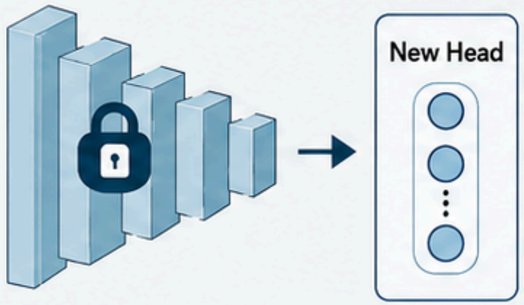
3 ImageNet-pretrained backnones were used

EfficientNetV2S
Accuracy

ResNet50
Classic Baseline

MobileNetV3Large
Speed

Phase 1 — Feature Extraction



- Backbone frozen
- Train new head only
- Learning rate: **1e-3**

Phase 2 — Fine-Tuning

- Unfreeze last N layers
  - EfficientNetV2S: 100
  - ResNet50: 30
  - MobileNetV3Large: 50
- Learning rate: **1e-5**
- 100× lower than Phase 1

From Frozen Features to Classical ML

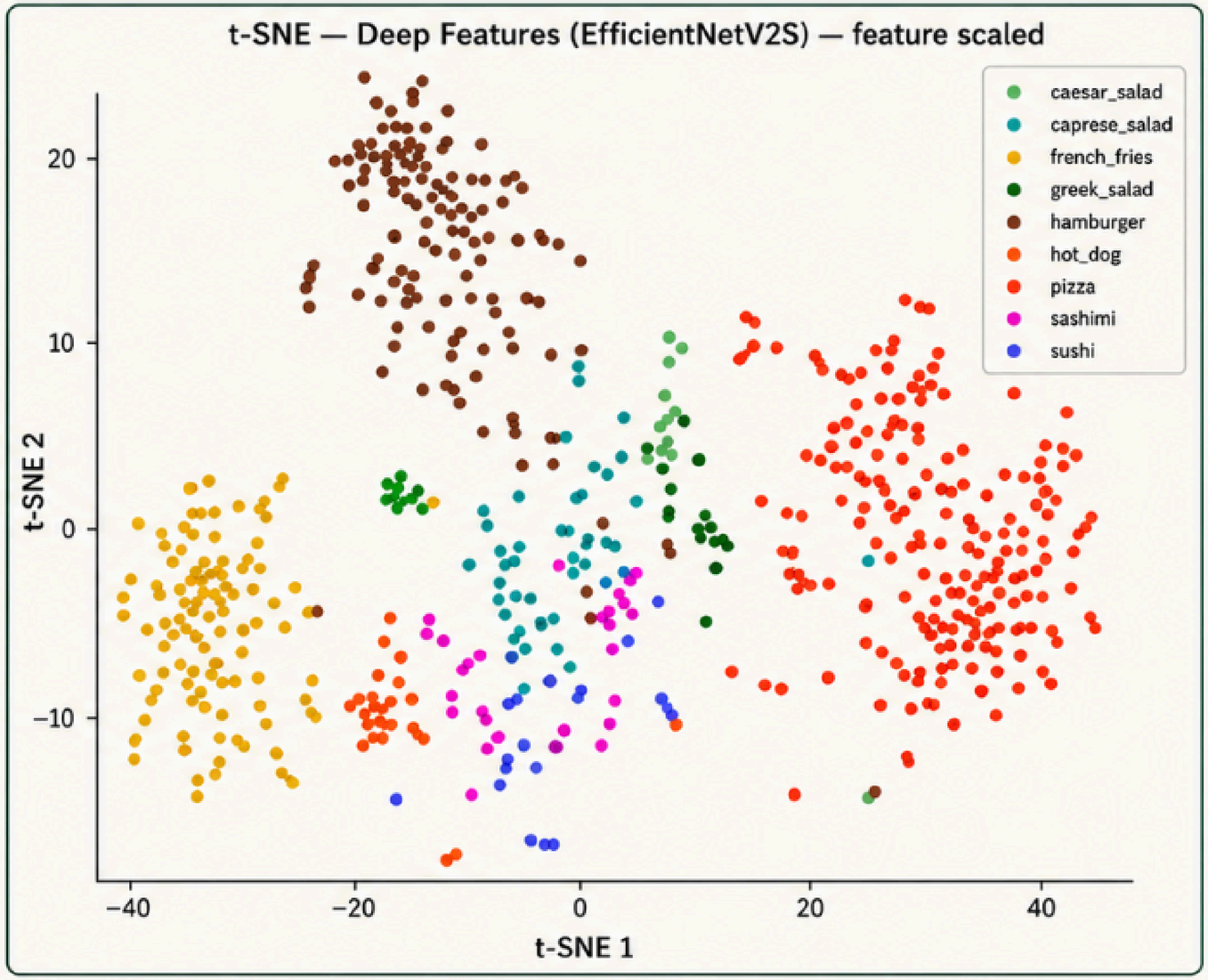
Why we tested ML classifiers

- **Third step:** we trained classical ML classifiers on top of the **best backbone** from the previous stage.
- From frozen **EfficientNetV2S** features, each image was represented by a **1280-d vector.**
- We used **PCA** for dimensionality reduction, then **t-SNE** to visualize the features in 2D and keep similar images close together.
- Some classes already formed **clear clusters** — for example **pizza** — suggesting that the frozen feature space was already informative.

We then tested

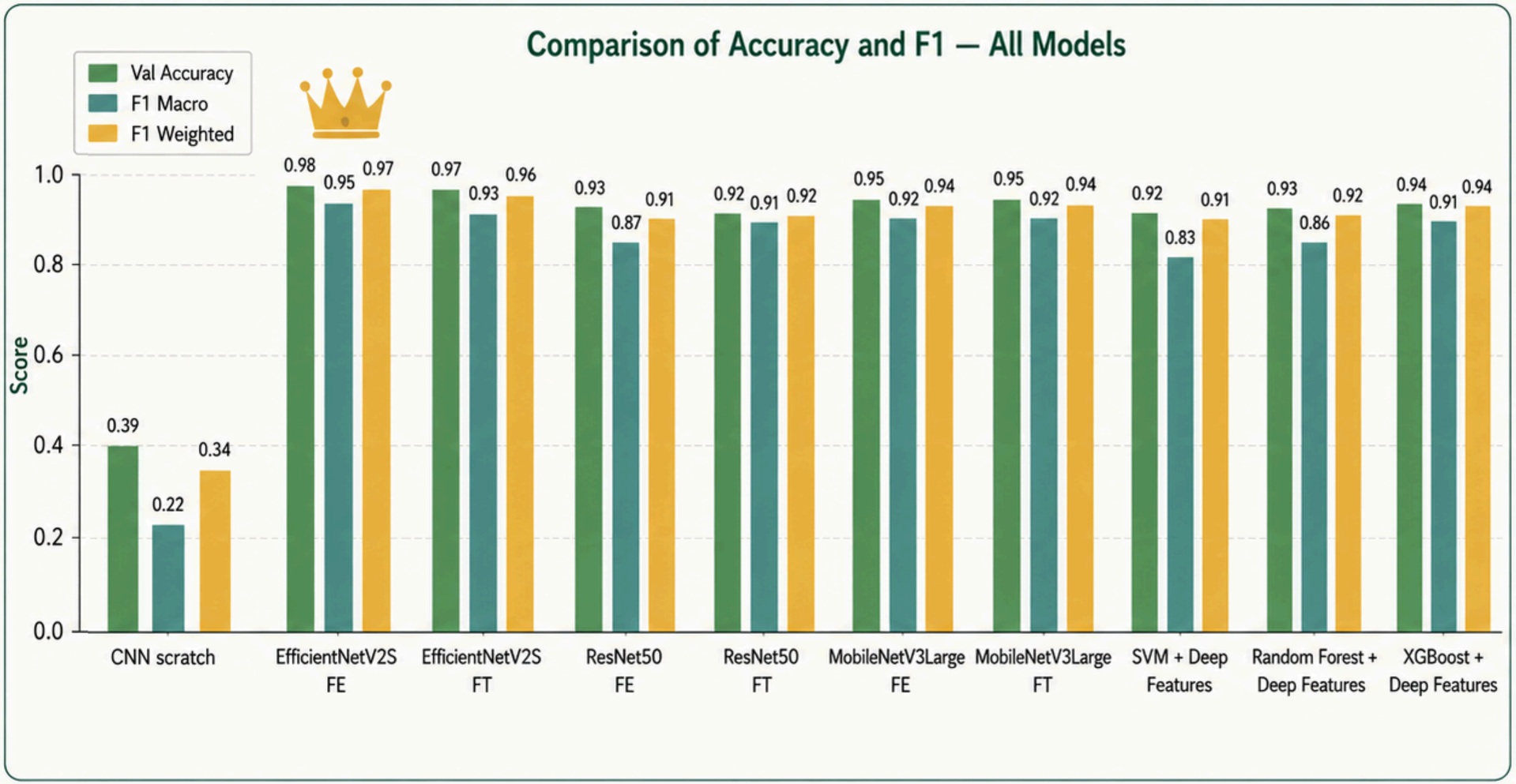
SVM | **Random Forest** | **XGBoost**

Goal: check whether simple classifiers on top of pretrained features could perform competitively.



Model Selection: Validation Results & Final Performance

Selection criterion: Macro F1 (not accuracy) — the dataset is imbalanced, so Macro F1 weights all 9 classes equally.



Final Test Set — EfficientNetV2S FE

Evaluated once on the held-out test set.

Accuracy	**96.2%**
F1 Macro	**0.86**
Top-3	**98.5%**

Key insight

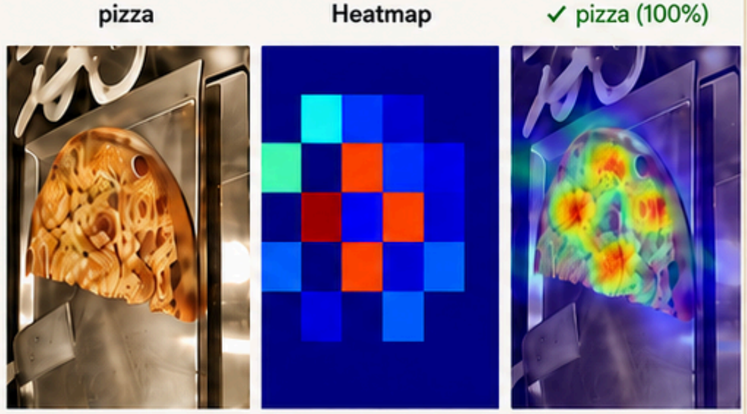
Fine-tuning did not help. With only 633 training images, unfreezing extra layers likely added overfitting. Also, some classes overlap with ImageNet, so frozen features were already task-relevant.

XAI: What does the Model Look At?

We wanted to understand which visual cues the model relies on.
To inspect both correct decisions and mistakes, we used **Grad-CAM** and **LIME**.

1 Grad-CAM — Correct Prediction

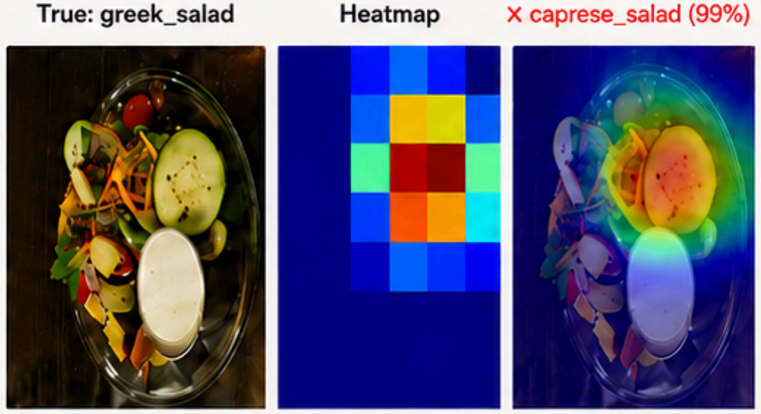
pizza (100%)



On correct predictions, the highlighted regions fall on the food itself, showing that the model focuses on relevant visual structure rather than the background.

2 Grad-CAM — Error Case

greek_salad → caprese_salad (99%)



Here the confusion is visually plausible: the model pays attention to the round white element and nearby salad components, which resemble cues associated with caprese salad.

3 LIME — Error Case

greek_salad → caprese_salad (99.2%)

LIME confirms the same pattern from a different perspective, highlighting the regions that most strongly support the wrong prediction.

Limitations - Future Works - Takeaway

Limitations

- Computational constraints prevented k-fold cross-validation, limited the CNN depth, and ruled out formal statistical tests.
- The dataset is small, which makes minority-class metrics noisy and less stable.
- Neural-network hyperparameters were set manually, while classical ML models used grid / random search.

Future Work

- **Data-related:** collect more data and expand the number of food categories.
- Run formal statistical significance tests; in particular, **McNemar's test** would help determine whether the observed performance differences are truly significant or just due to sampling variability.

Takeaway

1. Transfer learning clearly outperformed a CNN trained from scratch on this small dataset.
2. More adaptation is not always better: fine-tuning a pretrained backbone can hurt performance when data are limited.
3. Keeping finer label granularity can be the better choice: if merging similar classes brings little performance gain, the more informative class structure is worth preserving.

Demo Interface

We also built a simple interface to test the model on new photos taken by us and to make the system more accessible to a wider audience.

link:https://huggingface.co/spaces/IlariaCesa/food-FDL

THANK YOU

for your attention

Appendix - Curiosity

"**Why did fine-tuning perform worse, or the same, as feature extraction?**"
Because our **training set is very small** — only 633 images. When we unfreeze many layers, we add a lot of trainable parameters, and the **model starts fitting the noise** in the training data instead of learning real patterns. This is overfitting.
There is also a **second reason**: some of our **classes** — like pizza, hot dog, and hamburger — already **exist in ImageNet**, so the frozen backbone already had very good features for our task. Fine-tuning had little room to improve, and it introduced noise instead.

"Why didn't we use cross-validation?"
Cross-validation would require **retraining** each model **multiple times**. In our case, we have 3 backbones × 2 phases plus a CNN, each taking many minutes on **Colab's free GPU**. Doing 5-fold cross-validation would have multiplied the training time by 5, which was not feasible. Instead, we carefully built our split — stratified and group-aware — to make sure it was as representative as possible of the full dataset.

Appendix - Curiosity

"Why is the macro F1 lower on the test set (0.86) than on the validation set (0.93)?"

Mainly because some classes have **very few test examples.** Greek salad has only 3 images in the test set. If the model gets one wrong, the **recall** for that class **drops** dramatically, which **pulls the macro F1 down**. This is not necessarily a sign of overfitting — it is a consequence of having too few examples per class to get a stable estimate of performance.

"Why didn't we add warm-up to the fine-tuning phase?"

Warm-up gradually increases the learning rate at the beginning of training, to avoid large gradient updates before the optimizer has stabilized. We didn't use it because **our fine-tuning learning rate** was already **very conservative** — 1e-5, **one hundred times smaller than the feature extraction phase.** We also had **EarlyStopping** and **ReduceLROnPlateau** as safety mechanisms. On a small dataset, the impact of the first few batches is limited anyway.